altairsim — Migrating from AltairZ80 and z80pack

2026-09-06 · 4d137d1

This guide is for someone who already runs one of the two best-known open S-100 / Altair simulators — **AltairZ80** (part of the SIMH / Open SIMH project) or **z80pack** (Udo Munk) — and wants to know, honestly, what changes when you move to **altairsim**: what carries over untouched, what has a direct equivalent under a different name, and what you would give up.

It is written to be even-handed. AltairZ80 and z80pack are mature, well-made simulators that do things altairsim does not, and those are called out plainly. If your work depends on one of them, this guide will tell you so rather than talk you out of it.

Facts about the other two simulators here were checked against their own repositories and documentation as of September 2026 (Open SIMH `AltairZ80/`, `doc/altairz80_doc`; z80pack `master`, release 1.38). Where a detail could not be confirmed first-hand it is marked.

Why altairsim

The rest of this document is a fair, side-by-side comparison. This one section is not — it is the case *for* altairsim, stated plainly. The honest trade-offs are in §2 and §9; read those too.

- **It is the faithful one.** altairsim models the machine at the level of the S-100 bus and the cards plugged into it. A board decodes real ports and memory, cards can contend for the bus, the interrupt lines are actual wires, and `IN` / `OUT` run real cycles. Where the others treat a peripheral as a software channel, altairsim treats it as a card on the backplane — and it tells you when your machine is wired wrong (`SHOW BUS`).
- **A machine is a file you can mail.** One readable TOML file plus the disks beside it *is* the machine. Paths inside resolve relative to the file, so you copy the folder — or send it to someone — and it boots unchanged. No `.ini` to re-path, no `drivea.dsk` linking, no memory-map index to remember.
- **The debugger is always on, and it *is* the front panel.** `RUN`, `EXAMINE` and `DEPOSIT` are the panel switches; `HISTORY`, `TRACE`, `BREAK … IF`, symbolic disassembly, `STEP` / `NEXT` and `SNAPSHOT` / `RESTORE` are there the instant you launch — no rebuild, no flag.
- **You can hand it to a program — or an AI.** `--mcp` lets software drive a *running* guest: type at it, wait for what it prints, react. `-s` / `-x` give a shell-testable exit status for CI and Makefiles. Nothing in SIMH or z80pack reaches this level of automation.
- **It reads the disks you already have.** Both common 8-inch formats — the 337,568-byte hard-sector image and the 256,256-byte soft-sector image — mount directly, so your existing library comes with you (§4).

- **The widest 8-bit CPU set.** 8080, Z80, 8085 and the Motorola 6800 (a complete Altair 680b).
- **Install the same board as many times as you like.** Need three serial cards, or two disk controllers? Add `sio0`, `sio1`, `sio2` — each a distinct card at its own ports. SIMH models each peripheral as a fixed device (its console is a single 2SIO), so you cannot freely add more copies; altairsim's backplane takes as many of any board as the bus has room for.
- **One binary, no dependencies.** C++20 and CMake, and that is all; SDL3 is detected and used when present but never required. Unzip it and run.
- **It is documented and maintained.** A full user manual, a developer guide, and a command reference generated from the binary itself, so it cannot drift from the program.

If none of that outweighs a missing 8086, a graphical front panel, or an IMSAI 8080 — the things altairsim does *not* do — then §9 is the honest place to find that out before you switch.

---

1. The three projects at a glance

	AltairZ80 (SIMH)	**z80pack**	**altairsim**
What it is at heart	A general S-100 / CP-M-era hardware simulator that boots as an Altair	Faithful **front-panel** recreations of specific classic machines, plus a headless CP/M host	A MITS Altair 8800 / S-100 simulator focused on bus- and board-level fidelity
License	MIT-style	MIT	(see repository `LICENSE`)
CPUs	8080, Z80, **8086**	8080, Z80	8080, Z80, **8085**, **6800** (Altair 680b)
Machines it presents	Altair + many alternate S-100 configs (IMSAI, North Star, Cromemco, CompuPro, Vector Graphic, ADC, SCP, N8VEM…)	Altair 8800, **IMSAI 8080**, **Cromemco Z-1**, a generic CP/M host (`cpmsim`), Intel MDS-800, Mostek SBC, Pico	Many built-in machines (Altair variants, Sol-20, SD Systems SBC, Tarbell, iCOM, Cromemco FDC, CompuPro SS1, S100Computers, Altair 680b…)
Graphical front panel	No panel window (real Dazzler/VDM-1 **video** via SDL)	**Yes** — photoreal lights & switches (X11/OpenGL or SDL2; 2D/3D), plus a web panel on some systems	No panel window; the **monitor is the panel** (EXAMINE / DEPOSIT / RUN are the switches). Optional SDL3 for VDM-1 / Dazzler / video terminals
How you describe a machine	`.ini` command script (`SET` / `ATTACH`)	Per-system `conf/system.conf` + `[MEMORY n]` map sections + `disks/` directory	One **TOML machine file** = a self-contained, copyable directory
Debugger	SIMH `EXAMINE` / `DEPOSIT` / `BREAK`, `SAVE` / `RESTORE`	"ICE" monitor (off by default on the panel machines; needs a rebuild to enable)	Always-on monitor: `BREAK …` `IF`, `HISTORY`, `TRACE`, symbols, `STEP` / `NEXT`, `SNAPSHOT`
Scripting / automation	`.ini` do-scripts, `EXPECT` / `SEND`, Remote Console	Shell-script wrappers, socket-attached SIO ports	**`DO` a file of commands** (or `-s` at startup) — a shell-testable exit status; and **`--mcp`** (drive a live guest over JSON-RPC)
Networking	`NET` device (CP/NET, CPNOS)	—	TNFS disk mounts; serial over TCP sockets
Build	CMake (also legacy make / VS)	`make` (+ X11 or SDL2)	CMake, C++20, optional SDL3; a plain `make` convenience build

None of these rows is a verdict. Read §2.

2. What each simulator does best

Migrating is only sensible if the target does what you need. Here is where the other two are genuinely stronger, stated first so the rest of the guide is in context.

AltairZ80 (SIMH) is stronger when you need…

- **The Intel 8086.** AltairZ80 emulates a real 8086 (`SET CPU 8086`), so 8086-era S-100 software — e.g. Seattle Computer Products boards — runs. **altairsim has no 8086 and no plans stated here for one.** If your target software is 8086, stay on SIMH.
- **Breadth of vendors in one binary.** AltairZ80's device catalog spans CompuPro, Cromemco, North Star, IMSAI (FIF), Vector Graphic, Advanced Digital, Micropolis, Seattle Computer Products, N8VEM and more. altairsim's catalog is large but MITS-centred, with a curated set of third-party boards; some machines SIMH models have **no** altairsim equivalent (see §9).
- **Simulated networking.** The `NET` device runs CP/NET and CPNOS across simulated machines. altairsim has nothing equivalent.
- **The SIMH ecosystem.** The same command language (`EXAMINE`, `DEPOSIT`, `BREAK`, `SAVE`, `DO`, `EXPECT`, `SEND`, Remote Console) is shared with dozens of other SIMH simulators (PDP-11, VAX, PDP-10…). If you already live in SIMH, that transfer of skill is real and altairsim does not offer it.

z80pack is stronger when you need…

- **A real front panel.** z80pack's headline feature is a hardware-faithful **graphical front panel** — the actual lights and toggle switches — for the Altair 8800, IMSAI 8080 and Cromemco Z-1, with 2D and 3D models depending on the machine, and a web-based panel for some of them. **altairsim has no front-panel window at all.** If you want to flip the switches and watch the address LEDs, z80pack is the one that shows you that.
- **The IMSAI 8080 and Cromemco Z-1 as machines.** z80pack ships them as first-class systems. altairsim has Cromemco *disk controllers* (16FDC/64FDC boot CDOS) and a Sol-20, but **no IMSAI 8080 machine and no Cromemco Z-1 front panel.**
- **A dead-simple headless CP/M host.** `cpmsim` (`./cpm22`, `./cpm3`) boots straight to `A>` with stock disks and host↔guest file tools (`cpmr` / `cpmw`, `cpmsend` / `cpmrecv`). It is a very low-friction way to just *use* CP/M.
- **Minimal dependencies / very small hosts.** A compact `make`-based C codebase (optional X11 or SDL2), maintained since 1987, that even runs bare-metal on a Raspberry Pi Pico.

altairsim is stronger when you want…

This is where altairsim is built to win, and several of these have no equivalent in either other simulator:

- **Hardware fidelity at the bus and board level — the strictest of the three.** Boards decode real port and memory ranges, and a mis-wired machine is *diagnosable*, not silently wrong: `SHOW BUS` reports address collisions, the full interrupt wiring, and mis-strapped cards, and `IN` / `OUT` run real bus cycles with real side effects. Where the others simulate a device as a software channel, altairsim simulates the *card on the backplane*. The design goal is that no guest can tell it from the metal.
- **Self-contained, portable machines — copy the folder, it boots.** A machine is one readable TOML file plus the disks beside it. Paths inside it resolve relative to the file, so you can mail a machine to someone and it runs unchanged — no `.ini` to edit for local paths, no `disks/drivea.dsk` linking dance, no memory-map index to remember. `SHOW PATHS` shows every base at a glance.
- **An always-on debugger that *is* the front panel — no rebuild, ever.** `RUN`, `EXAMINE` and `DEPOSIT` are the panel's switches, and `HISTORY` (a live ring of recent bus cycles), `TRACE`, `BREAK … IF <condition>`, symbolic disassembly, `STEP` / `NEXT`, and `SNAPSHOT` / `RESTORE` are all there the instant you launch. z80pack's ICE monitor is off by default and needs a source edit and rebuild to switch on; altairsim's is simply always on, and richer.
- **First-class automation, including AI — genuinely unique here.** `altairsim -s script` and `-x cmd` give you a shell-testable exit status for CI and Makefiles, and `--mcp` lets a program *or an AI agent* drive a **running** guest — type at it, wait for the output it prints, react — over plain JSON-RPC. Neither SIMH's `EXPECT` / `SEND` nor z80pack's shell wrappers offer anything at this level. See `docs/manual/mcp.md` and `docs/DRIVING-WITH-AI.md`.
- **The widest CPU line-up.** 8080, Z80, 8085 *and* the Motorola 6800 (a full Altair 680b) — more than either other simulator on the 8-bit side.
- **Multiple instances of any board.** Because a board is a card on the bus, you can plug in as many of the same type as the address map allows — several 2SIOs on different port ranges, more than one disk controller, and so on. SIMH exposes each peripheral as a fixed device and does not let you freely instantiate additional copies of it; altairsim's model does this for free.

---

3. The mental-model shift

The single biggest change is **how you describe a machine.**

- **SIMH** describes a machine as a *script of commands* — an `.ini` file (or the auto-loaded `altairz80.ini`) full of `SET`, `ATTACH` and `BOOT` lines that are executed in order to configure and start the machine.
- **z80pack** describes a machine as a *directory of one system* — a `conf/system.conf` with key/value settings and `[MEMORY n]` map sections, a `disks/` folder, and shell scripts that hard-link the right image into a fixed drive name (`disks/drivea.dsk`) and launch the binary.
- **altairsim** describes a machine as a *declarative TOML file* — a backplane with `[[board]]` entries, each with its settings, and an optional `startup` block for what to do once it is built. It is a picture of the hardware, not a script to build it.

```
# a fragment of an altairsim machine file
[[board]]
type = "8080"
id   = "cpu0"

[[board]]
type = "2sio"
id   = "sio0"

[[board]]
id = "dsk0"                      # the controller
  [[board.drive]]
  unit  = 0
  mount = "cpm22b23-56k.dsk"     # relative to THIS FILE — copy the folder, it still boots
```

Three consequences worth internalising:

1. **You usually do not write a boot script.** Where a SIMH session ends with `boot dsk`, an altairsim machine either boots from its PROM at reset or names the boot in `startup`. From the monitor, starting the machine is `RUN` (optionally `RUN <addr>`), which is the panel's switch.
2. **A board is a real object on a bus.** In SIMH a "device" is largely a software channel; in altairsim a board *decodes ports and memory*, and two boards fighting over port 08 is a diagnosable fault (`SHOW BUS CONTENTION`), not a silent last-writer-wins.
3. **There is no global `SET CPU Z80`.** The CPU is a board. You pick a machine whose CPU board is a `z80` (the built-in `z80` machine, or any machine file with `type = "z80"`), or you `BOARDS ADD z80 cpu0` on an empty backplane (`altairsim -n`).

To see any built-in machine as an editable file — the fastest way to learn the format — save one out and open it:

```
$ altairsim -x 'CONFIG SAVE mine.toml' default
$ altairsim mine.toml
```

---

4. Disk images: what ports directly, and what does not

This is the part with the best news and the sharpest trap, so read it carefully. There are **two different "8-inch" image formats** in circulation, and they are not interchangeable.

The hard-sectored 88-DSK format — ports directly

The genuine MITS 88-DCDD / 88-DISK geometry is **77 tracks × 32 sectors × 137 bytes = 337,568 bytes** (the 137 includes the sync/header and checksum bytes around each 128-byte payload). This is:

- what altairsim's `dcdd` controller mounts as its `8in` format,
- what SIMH's `DSK` device attaches, and
- what z80pack's *altairsim* MITS 88-DCDD path (`boot-mits`) uses.

A 337,568-byte hard-sector image from SIMH or z80pack mounts in altairsim unchanged. No conversion, no flags. altairsim probes the geometry from the byte count, so there is nothing to declare:

```
altairsim> MOUNT dsk0:drive0 cpm2.dsk
```

(XMODEM-padded copies — e.g. 337,664 bytes — are accepted too; the probe allows the padding.)

> **A boot-ROM difference worth knowing.** AltairZ80 boots by default through a *modified* Altair disk boot loader (`SET CPU ALTAIRROM`, on by default). A stock 88-DSK image that expects the genuine MITS DBL may not boot under SIMH until you `SET CPU NOALTAIRROM` and load DBL yourself. altairsim uses the **real DBL PROM** (`builtin:dbl`), so a genuine image boots the way it did on the hardware — one fewer thing to reconcile when you carry an image across.

The 128-byte soft-sector format — also ports directly, onto a soft-sector controller

The other common 8-inch image is the IBM-3740-style **single-density soft-sector** (SSSD) layout: **77 tracks × 26 sectors × 128 bytes = 256,256 bytes**, storing the sector payloads only (no sync/header/checksum framing). This is a genuinely *different* format from the hard-sectored MITS image above — different sectoring, different geometry, no framing bytes — not a smaller version of it. It is what z80pack's `cpmsim` and `imsaisim` use, what the Tarbell soft-sector path uses, and what most `cpmtools`-produced images are.

altairsim mounts it directly too — just on a soft-sector controller, not the hard-sector `dcdd`. The 256,256-byte SSSD geometry is native to altairsim's `tarbell`/`tarbelldd`, `versafloppy`, `icom`, and `16fdc`/`64fdc` boards, and the geometry is probed from the byte count exactly as for the hard-sector format:

```
altairsim> MOUNT dsk0:drive0 cpm-sssd.dsk       # dsk0 here being a tarbell/versafloppy/etc.
```

The **one trap** is mixing them up: a 256,256-byte SSSD image will not work on the hard-sector `dcdd` (which reads 337,568-byte 8-inch and 8,978,432-byte 8 MB images), and a 337,568-byte hard-sector image will not work on a soft-sector board. Match the image to the controller kind — `SHOW BOARDS` names each board's type — and altairsim does the rest. As always, whether a given disk then *boots* depends on its filesystem and BIOS matching the machine you mount it in, the same as it did on the simulator you came from.

If instead you only need the **files** off a disk (any format, including formats no altairsim board reads), boot a CP/M machine and pull them in with the **Host Bridge** (`R.COM`/`W.COM`, sandboxed to a host directory) — altairsim's intended path, and it avoids `cpmtools` entirely. An `.imd` ImageDisk file is handled automatically: `MOUNT foo.imd` reads it, writes a raw sibling `.dsk`, and mounts that.

Hard disks and other media

SIMH's `HDSK` device and its many `FORMAT=` presets do not map one-to-one. altairsim's own hard disk is the **88-HDSK** "**Datakeeper**" (`hdsk`, fixed geometry: 406 cyl × 2 × 24 × 256), which takes a full-size linear image you supply — there is no blank-and-format step for it. For very large CP/M volumes altairsim also has an 8 MB floppy medium (`fdc8mb`) on the stock `dcdd`, and CompactFlash/SD card images on the `dualide`/`dualsd` boards. Match the controller to the image you have; `SHOW MOUNTS` and `SHOW PATHS` will tell you what landed where.

---

5. Command reference — SIMH → altairsim

> ### ⚠ The one that will trip your fingers: `D` and `E` are swapped
>
> In SIMH, `D` is **DEPOSIT** and `E` is **EXAMINE**. In altairsim, **`D` is DUMP** (read a page, harmless), **`DE` is DEPOSIT**, and **`EX` is EXAMINE**. This is deliberate — the safe, most-typed command gets the single letter, and there is no `EXIT`, so `E` is free for `EDIT`. Until the new habit sets in, a reflexive SIMH `D 100 3E` in altairsim will *dump* page 0100 instead of writing a byte. To write a byte: `DE 100 3E`.

You did in SIMH	In altairsim	Notes
`SET CPU 8080` / `Z80`	pick a machine with that CPU board, or `BOARDS ADD z80 cpu0`	CPU is a board, not a global mode
`SET CPU 8086`	—	not supported (see §2)
`SET CPU 64K`	a `memory` board region in the machine file	memory is a board
`SET CPU BANKED`	a `bankmem` board (`vector` / `cromemco64kz` / `northstar` / `expandoram2`)	banking is its own board
`ATTACH DSK cpm.dsk`	`MOUNT dsk0:drive0 cpm.dsk`	337,568-byte images port directly (§4)
`ATTACH HDSK0 hd.dsk`	`MOUNT hd0:drive0 hd.dsk`	geometry differs; see §4
`SET DSK0 WRTLCK`	`MOUNT … WP` (or `readonly = true` in the file)	`SHOW MOUNTS` marks it
`BOOT DSK`	`RUN FF00` (run the boot PROM)	a machine file's `startup` can do it on load
`EXAMINE 100-1FF`	`DUMP 100-1FF` (or `D 100`)	a range means exactly what it says
`EXAMINE PC` (one cell)	`EXAMINE 100` / bare `EX` to step	EXAMINE jams the PC and reads one byte
`DEPOSIT 100 3E`	`DE 100 3E`, or `EDIT 100` (interactive)	`EDIT` walks byte-by-byte
`DEPOSIT PC 1040`	`SET REG PC=1040` (or `RUN 1040`)	`RUN <addr>` = EXAMINE + RUN
`EXAMINE AF` / registers	`REGS`; `SET REG A=3F`	flags too: `SET REG CY=1`
`RUN` / `GO`	`RUN`	resumes; `RUN <addr>` sets PC first
`CONTINUE`	`RUN` (bare)	resumes from where STOP stopped
`STEP` / `STEP n`	`STEP` / `NEXT`	`NEXT` steps *over* CALL/RST
`BREAK 100` / `NOBREAK`	`BREAK 100` / `NOBREAK`	plus `BREAK … IF <cond>`
`SAVE f` / `RESTORE f`	`SNAPSHOT f` / `RESTORE f`	state, not machine shape — build the machine first
`SHOW CONFIGURATION` / `SHOW DEVICES`	`BOARDS`, `SHOW MACHINE`	`BOARDS` is the backplane
`SET REMOTE TELNET=n`	`CONNECT sio0:a socket:n` (console), or `--mcp`	different mechanisms for different jobs
`DO script.ini`	`DO script.ini` (at the prompt) or `altairsim -s script.ini` (at startup)	same command — one line at a time, as if typed; paths inside a `DO` file are relative to the file

`EXPECT` / `SEND`	`altairsim <machine> --mcp` (`run {input:…, until:…}`)	drive a live guest; see `docs/manual/mcp.md`
an `altairz80.ini` startup file	a TOML machine file (auto-loads `./altairsim.toml`), or a `DO` -able `.ini` of commands	the TOML is declarative; the `.ini` is the batch-of-commands path, closest to what you have

Stopping a running machine: in SIMH you press **Ctrl-E** to return to `sim>`. In altairsim the stop key is **Ctrl-E** as well — it presses the front-panel **STOP** switch — but note Ctrl-C belongs to the *guest* (CP/M reads it), which is exactly why the stop is Ctrl-E and not Ctrl-C.

---

6. Command reference — z80pack → altairsim

z80pack is configured more by files and flags than by an interactive command language, so the mapping is mostly config-to-config.

You did in z80pack	In altairsim	Notes
`./cpm22`, `./cpm3` (boot scripts)	`altairsim <cpm-machine>`	a machine file that mounts its disk and boots
`./altairsim` (front panel)	`altairsim default` then use the **monitor**	no panel window; `EXAMINE` / `DEPOSIT` / `RUN` are the switches
`-8` / `-z` (CPU select)	pick a machine with an `8080` / `z80` CPU board	
`-f <MHz>` (CPU speed; `0` = flat out)	`clock_hz` on the CPU board (`0` = flat out, the default)	same idea, per-board
`-x prog.hex` (load Intel HEX)	`LOAD prog.hex`	auto-detects HEX / S-record / binary
`-s` / `-l` (save/load core)	`SNAPSHOT` / `RESTORE`	
`-m <hex>` (memory fill)	`FILL <range> <byte>`	or a region default in the file
`conf/system.conf`	the TOML machine file	one file, self-contained
`[MEMORY n]` + `-M n` (map sections)	`memory` / `bankmem` boards with explicit regions	no section index; the boards *are* the map
link `disks/library/x.dsk` → `disks/drivea.dsk`	`MOUNT dsk0:drive0 x.dsk`, or name it in the file	no fixed-name linking dance
`sioN` socket routing	`CONNECT sio0:a socket:<port>`	listen or dial out; carrier modelled
`cpmr` / `cpmw`, `cpmsend` / `cpmrecv`	Host Bridge `R.COM` / `W.COM` (sandboxed)	altairsim's guest↔host file path
ICE monitor (`WANT_ICE`, rebuild)	the built-in monitor (always on)	`HISTORY`, `TRACE`, `BREAK … IF`, symbols
`-n` web front panel	—	no web panel; `--mcp` and socket consoles instead
SIO settings (`upper_case`, `strip_parity`…)	`[console]` transforms (`upper`, `strip7in/out`, `crlf`…)	in altairsim these belong to the console, not the card

7. Worked migration — from a SIMH `.ini`

A very common AltairZ80 CP/M session looks like this:

```
; altairz80.ini
set cpu 8080
set cpu 64k
attach dsk cpm2.dsk
attach dsk1 apps.dsk
boot dsk
```

The equivalent altairsim machine file (`cpm.toml`, kept in a folder beside its disks):

```
[machine]
name    = "cpm"
startup = ["RUN FF00"]        # throw the RUN switch on load -> straight to A>

[[board]]
type = "8080"
id   = "cpu0"

[[board]]
type = "memory"
id   = "mem0"
  [[board.region]]
  type = "ram"
  at   = 0x0000
  size = "56K"                # RAM 0000-DFFF; the boot PROM lives up at FF00
  [[board.region]]
  type  = "rom"
  at    = 0xFF00
  mount = "builtin:dbl"       # the DBL boot PROM, shipped in the binary
                              # (a custom PROM would be mount = "yourprom.hex")

[[board]]
type = "2sio"
id   = "sio0"

[[board]]
type = "dcdd"
id   = "dsk0"
  [[board.drive]]
  unit  = 0
  mount = "cpm2.dsk"          # your 337,568-byte SIMH image, unchanged
  [[board.drive]]
  unit  = 1
  mount = "apps.dsk"
```

Then it boots on launch, because `startup` runs the boot PROM for you:

```
$ altairsim cpm.toml          # -> A>
```

(Leave the `startup` line out and you get the monitor instead; `RUN FF00` boots it by hand.)

Even closer to your `.ini`: a `DO` script. If you would rather keep a *batch of commands* than a declarative file, altairsim's `DO` command runs one — one line at a time, exactly as if you typed it. It is the near-literal translation of the `.ini` above, command for command:

```
; cpm.ini
MACHINE default              ; an 8080 with 56K + the DBL boot PROM  (SIMH "set cpu 8080" / "64k")
MOUNT dsk0:drive0 "cpm2.dsk" ; attach dsk
MOUNT dsk0:drive1 "apps.dsk" ; attach dsk1
RUN FF00                     ; boot dsk
```

Run it either way — at startup from the shell, or at the `altairsim>` prompt:

```
$ altairsim -s cpm.ini          # from the command line -> A>, exits with the script's status
altairsim> DO cpm.ini           # at the monitor prompt, any time
```

`MACHINE default` is the whole hardware in one line (`MACHINE none` gives you an empty backplane to `BOARDS ADD` onto instead); paths inside a `DO` file are relative to the file, so the folder copies and still runs. The TOML above is still the idiomatic, shippable form — but the `.ini` is the shortest bridge from what you have. The packaged `examples/` folders carry a `.ini` beside their `.toml` to show the two side by side.

The fastest way to get the boards and PROM exactly right is not to type them from scratch: `altairsim -x 'CONFIG SAVE cpm.toml' default` writes the shipped `default` machine (8080, DBL PROM, 88-DCDD) out as a working file to add your disks to, and `examples/cpm/cpm22-buffered.toml` is a complete, booting CP/M machine you can copy and point at your own image.

If your `.ini` ended with `EXPECT`/`SEND` automation, that becomes an MCP session instead:

```
$ altairsim cpm.toml --mcp
# JSON-RPC on stdio: initialize → run {from:0xFF00} → run {input:"DIR\r", until:"A>"}
```

---

8. Worked migration — from a z80pack system

A z80pack Altair boot uses a script that stages a disk and selects a memory map:

```
# altairsim/boot-mits  (z80pack)
./altairsim -M 3 $*          # -M 3 = the memory map with the external boot ROM
# with disks/mits_a.dsk hard-linked into place beforehand
```

In altairsim there is no separate stage-and-link step and no map index — the machine file *is* the map, and it names its own disk:

```toml
[machine]
name    = "8800"
startup = ["RUN FF00"]        # what -M 3's boot ROM did for you

[[board]]
type = "8080"
id   = "cpu0"

[[board]]
type = "memory"
id   = "mem0"
  [[board.region]]
  type = "ram"
  at   = 0x0000
  size = "56K"
  [[board.region]]
  type  = "rom"
  at    = 0xFF00
  mount = "builtin:dbl"       # altairsim ships DBL; no need to bring z80pack's ROM
                              # (a custom boot PROM would be mount = "yourprom.hex")

[[board]]
type = "2sio"
id   = "sio0"

[[board]]
type = "dcdd"
id   = "dsk0"
  [[board.drive]]
  unit  = 0
  mount = "mits_a.dsk"        # was disks/mits_a.dsk
```

```
$ altairsim 8800.toml         # -> A>
```

If you relied on z80pack's **front panel**, this is where you feel the difference most: altairsim gives you the panel's *function* at the monitor rather than a window of lights. Reading and writing memory, jamming an address into the PC, single-stepping and running are all there — `EX F800`, `STEP`, `RUN` — but there is nothing to look at but text. If the visual panel is the point of your setup, keep z80pack for that.

For the graphical boards z80pack's Altair can drive — the **VDM-1** and the **Dazzler** — altairsim has real equivalents (`vdm1`, `dazzler`) that open an SDL3 window, so that part of the experience does carry over on a build with SDL3.

---

9. What you give up (be honest with yourself before you switch)

- **The Intel 8086.** Only SIMH has it. 8086 S-100 software cannot run on altairsim.
- **A graphical front panel.** Only z80pack has one. altairsim's panel is the text monitor.
- **The IMSAI 8080 and Cromemco Z-1 as complete machines.** z80pack ships them; altairsim does not (it has Cromemco *disk controllers* and a Sol-20, but no IMSAI and no Z-1 panel).
- **Simulated networking (CP/NET).** Only SIMH. altairsim's networking is TNFS disk mounts and serial-over-socket, not a simulated CP/M LAN.
- **Specific boards/machines modelled elsewhere but not here.** North Star Horizon disk, Micropolis, IMSAI FIF, Intel MDS-800, Mostek SBC and others are in SIMH and/or z80pack with no altairsim counterpart. Conversely, altairsim models boards *they* do not — check the **Boards** chapter (`SHOW BOARDS` lists every type) before assuming a gap in either direction.
- **The wider SIMH skill set.** If you drive PDP-11s and VAXen with the same commands, that shared muscle memory does not come with altairsim.

If none of those is load-bearing for you, the move buys you self-contained machine files, a bus-accurate board model, an always-on debugger, and `--mcp`.

---

10. A migration checklist

1. **Confirm the CPU.** 8080/Z80/8085/6800 → fine. **8086 → stop, stay on SIMH.**
2. **Confirm the machine exists.** `altairsim --list`. Need an IMSAI 8080 or a graphical panel → z80pack is the better home.
3. **Sort your disks by size, then match the controller kind.** 337,568 bytes → a hard-sector board (`dcdd`). 256,256 bytes → a soft-sector board (`tarbell`, `versafloppy`, `icom`, `16fdc` / `64fdc`). Both mount directly; the only mistake is crossing them (§4).
4. **Start from a built-in.** `altairsim -x 'CONFIG SAVE mine.toml' <closest-machine>`, then edit — do not write the file from scratch.
5. **Relearn `D` / `E`.** `D` dumps, `DE` deposits, `EX` examines (§5).
6. **Port your automation.** `.ini` do-scripts → a `DO` file of the same commands (`DO x.ini` at the prompt, or `altairsim -s x.ini` at startup); `EXPECT` / `SEND` or z80pack shell wrappers → `--mcp`.
7. **Verify on a real boot.** `altairsim mine.toml`, `RUN`, reach `A>`, run a command. If it boots and behaves, you are migrated.

Where to read more

These ship beside this guide in the package:

- `QUICK-START.pdf` — boot CP/M in one command. The shortest path to a running machine.
- `altairsim-manual.pdf` — the User Manual: quick start, machines, disks, boards, MCP, file transfer. Its MCP chapter and `DRIVING-WITH-AI.md` cover driving a running guest programmatically.
- `altairsim-monitor.pdf` and `altairsim-debugger.pdf` — the `altairsim>` prompt and the built-in debugger, including the two deliberate breaks with SIMH in how commands rank and abbreviate.

Deeper still, in the **source repository** (https://github.com/deltecent/altairsim): `DESIGN.md` for the architecture and the reasoning behind the board/bus model, and `docs/cli-commands.md` for the full command-ranking rationale.